

Sapling DB - Modular Redesign

Developer migration guide: what changed and what you need to know

One offering-shaped 'courses' table and a 'users' mega-table became 8 bounded domains of typed tables with foreign keys, CHECK enums, a real term/semester entity, and one source of truth per fact. The schema landed as one source-of-truth set of migrations (0019-0028); the application code was rewired domain-by-domain on top of it.

Status: 790 unit tests pass (mocked) + 14/14 service checks pass against live staging data. Two real-DB bugs were found and fixed during staging validation (see Gotchas).

Branch: epic/db-modular-redesign | Migrations: 0019-0028 | Spec: docs/superpowers/specs/2026-06-23-db-modular-redesign-design.md

1. TL;DR - the mental model

Three ideas carry 90% of this change. Internalize these and most of the migration is mechanical:

- A course now has THREE identities. The abstract course (catalog: code, name, credits) is separate from a course_offering (that course taught in a specific term) which is separate from an enrollment (one user in one offering).
- The PUBLIC API still speaks the abstract course_id. Nothing on the wire changed - the frontend keeps sending one course id. The split is resolved server-side.
- Different domains key on different ids: the knowledge GRAPH stays on the abstract course_id (mastery is cumulative across terms); GRADEBOOK keys on enrollment_id; STUDY + ANALYTICS key on offering_id (per term).

Term / semester is now a real, orderable, date-ranged entity (the 'terms' table) instead of a free-text 'Spring 2026' string. 'Current term' is date-derived.

The resolver you will use: [services/academics.py](#)

Never hand-roll the course<->offering<->enrollment lookups. Use these:

```
current_term()          -> the term row whose date range
                        contains today (latest-term fallback)
list_terms()            -> all terms (backs GET /api/semesters)
resolve_offering(course_id, term_id=None, *, create=False)
                        -> the offering for (course, term);
                        defaults to current term
offering_course_id(offering_id) -> the abstract course_id (graph bridge)
user_offering_ids_for_course(uid, cid) -> a user's offerings of a course
term_for_offering(offering_id) -> the term row (labels)
```

2. The id model - read this first

This is the single most common source of confusion. When you have a 'course id', ask which of the three it is:

Identity	Table	Means	Used by
abstract course	courses	the catalog entry (CS101)	graph, API boundary, catalog
offering	course_offerings	CS101 taught in Fall 2025	study, analytics, gradescope
enrollment	enrollments	a user taking that offering	gradebook (grades, GPA)

Resolution direction: enrollment.offering_id -> course_offerings.course_id -> the abstract course. To go the other way (user enrolling in 'CS101'), resolve_offering() picks the current-term offering (creating it if the catalog lacks one).

Why the graph stays abstract: a student's mastery of 'Recursion' should accumulate whether they took CS101 last fall or this spring. Class analytics, by contrast, are per-offering (one class section in one term), so they key on offering_id.

3. Table rename cheat-sheet (old -> new)

If your code touches the left column, it must change. The mocked test suite will NOT catch a missed rename if you also update its mocks - verify against the real schema.

Old	New	Note
courses (offering-shaped)	course_offerings	+ new abstract 'courses' table
courses.semester (text)	terms (entity)	via course_offerings.term_id
user_courses	enrollments	course_id -> offering_id
course_categories	gradebook_categories	keyed on enrollment_id
assignments (user+course)	assignments	keyed on enrollment_id
course_concept_stats	offering_concept_stats	keyed on offering_id
course_summary	offering_summary	keyed on offering_id
users.name / profile cols	user_profiles	1:1; name stays encrypted
graph_nodes.mastery_events	node_mastery_events	JSON blob -> append-only rows
feedback / issue_reports	(same)	int PK -> text PK + FKs

Reading a user's display name

users.name and the profile fields moved to user_profiles (encrypted). Do NOT select users.name - it no longer exists. Use the helper:

```
from services.profiles import get_display_name, get_display_names
name = get_display_name(user_id) # decrypts user_profiles.name
names = get_display_names([id1, id2, ...]) # bulk, for lists/leaderboards
```

4. What to change in your code

- Querying enrollments: table('enrollments'), filter/insert on offering_id (not table('user_courses') / course_id).
- Gradebook: resolve (course_id, semester) -> the user's enrollment via services.academics, then key categories + assignments on enrollment_id. Per-term GPA is per offering; cumulative GPA is credit-weighted across a user's offerings.
- Study artifacts (documents, notes, sessions, flashcards, study_guides): key on offering_id. Resolve from the abstract course + current term when you only have a course.
- Graph writes still go through services.graph_service.apply_graph_update, which now appends node_mastery_events rows (atomic) instead of rewriting a JSON blob.
- Analytics: read/write offering_concept_stats + offering_summary by offering_id; there is no free-text 'semester' column anymore.
- All DB access still goes through db/connection.py::table(). Encryption boundary is unchanged: encrypt_if_present at write, decrypt_if_present / decrypt_numeric at read.

Encryption (unchanged boundary, new home for names)

Encrypted (stays TEXT): user_profiles name/first_name/last_name/bio/location, users.email, oauth tokens, messages.content, room_messages.text, sessions.summary_json, documents.summary/concept_notes, notes.title/body/last_summary, gradebook points_possible/points_earned, gradescope credentials. Decrypt numeric points with decrypt_numeric before computing grades.

5. Migrations and ops

- Migrations are append-only and immutable once applied. Never edit an applied file - add a new NNNN_ file (we are at 0028). The raw-DDL runner is db/migrate.py.
- The db scripts read env via python-dotenv's load_dotenv() (loads .env) - they do NOT auto-read .env.staging. Always wrap staging/prod commands explicitly.

Apply schema + seed to staging

```
# from backend/, with the staging dotenv CLI on PATH:
dotenv -f .env.staging run -- python -m db.migrate      # apply pending 00NN
dotenv -f .env.staging run -- python -m db.seed_staging # idempotent demo data
```

Promote to production (cutover)

```
SUPABASE_DB_URL=<prod-direct> python -m db.migrate --baseline # record 0001-0018
SUPABASE_DB_URL=<prod-direct> python -m db.migrate           # apply 0019+
```

- NEVER make schema changes in the Supabase dashboard / SQL editor - the repo is the only source of truth, or staging and prod silently diverge. (A 'NOTIFY pgrst, reload schema' cache reload is fine - it is not a schema change.)
- Prod has no user data (only the course catalog), so the catalog transform in 0020 is the only data migration; user tables are clean cutovers.

6. Validation status

Two layers. The mocked suite proves the code is internally consistent; the live-staging run proves the rewired services work against the real migrated schema + seeded data.

- Mocked unit suite: 790 passed (pytest, no DB).
- Live staging service checks: 14/14 passed against seeded demo data, including:

Service / function	Validated
academics resolver	terms, current_term, offering + enrollment resolution
graph_service.get_courses	4 enrollments incl. CS101 in TWO terms, each w/ term
graph_service.get_graph	9 nodes + subject roots + edges + tier stats
profiles.get_display_name	decrypts user_profiles.name -> 'Demo Student'
course_context (analytics)	offering-scoped aggregate computed + read back
graph_read tool	concepts returned weakest-first
learn course resolution	topic 'CS101' -> abstract course id
gradebook resolve + grade	CS101 Fall = (92.0, 'A-'): drop-lowest + decrypt points

7. Gotchas + bugs found during rollout

Seeding staging caught two real-DB bugs that 1000+ mocked tests could not - because mocks never hit the actual schema. This is exactly why we seed + smoke-test staging before the production cutover.

- 0028 (schema): 0020 renamed courses -> course_offerings and dropped the abstract columns but MISSED course_code, which stayed NOT NULL. Existing rows had it populated, so it was invisible until a NEW offering insert - which the app's add_course / resolve_offering(create=True) does. Would have broken enrollment in production. Fixed by dropping the vestigial column.
- Seed _exists_by: assumed every table has an 'id' column; user_profiles is keyed on user_id (no id). Fixed to select a filtered column.
- Known cosmetic follow-up: a course taken in two terms renders two subject-root hub nodes in the graph (one per enrollment). Harmless; de-dupe by abstract course is a small fix.

8. Open follow-ups

- Frontend (#260): surface term, add a SemesterChips picker from GET /api/semesters, and let users pick an explicit offering when a course spans terms.
- Gradescope code rewire (#265): the sync (on a teammate branch) must re-key onto enrollment_id per the new 0027 schema.
- Schools: courses.school free-text was retired; school_id is unpopulated, so 'school' surfaces blank until the schools table is populated/linked.
- Deploy the epic branch to staging and smoke-test the live HTTP app, then open the epic -> main cutover PR (docs refresh + green CI + prod promotion).

9. Command cheat-sheet

```
# run the mocked suite
cd backend && python -m pytest tests/ -q

# apply migrations / seed to staging (idempotent)
dotenv -f .env.staging run -- python -m db.migrate
dotenv -f .env.staging run -- python -m db.seed_staging

# confirm which DB an env points at (no secrets printed)
dotenv -f .env.staging run -- python -c \
    "import os;from urllib.parse import urlparse as u;\
    print(u(os.getenv('SUPABASE_URL')).netloc)"
```